DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Modelado, simulación y prueba en la tarjeta MAX1000 de un flip-flop con entradas síncronas de control.**	**CÓDIGO:** **ANP2**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Libre. Ejercicios.	TIPO:	No presencial	DURACIÓN:	110 minutos

CALENDARIO:	Previo a la sesión presencial S7.	REQUISITOS:	Conocimientos adquiridos en Electrónica 2. Modelado VHDL de circuitos secuenciales. ModelSim. Quartus Prime

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

Esta actividad está dividida en tres partes. En la primera modelará en VHDL el comportamiento de un circuito secuencial sencillo. En la segunda simulará, mediante la herramienta *ModelSim*, el modelo anterior, para lo cual deberá escribir el correspondiente *testbench* para el circuito. En la tercera parte empleará la herramienta *Quartus Prime* para sintetizar el circuito y probar el mismo sobre la placa MAX1000.

PARTE I. Modelo VHDL del Circuito

Deberá escribir el modelo de un circuito secuencial, llamado *dff_ser*, que es un *flip-flop* tipo *D* cuyas entradas y salidas son:

- Entrada de *reset* asíncrono *RST_an* activa a nivel bajo
- Entrada de reloj *CLK* activo en flanco de subida
- Entrada de *set* síncrono *SET_n* activa a nivel bajo.
- Entrada de habilitación *E* activa a nivel alto. *SET_n* no requiere *E*
- Entrada de *reset* síncrono *RST* activa a nivel alto. Esta entrada requiere *E* y es menos prioritaria que *SET_n*
- Entrada de dato *D*
- Salida de dato *Q*

Para ello (trabajando siempre dentro del directorio *ANP2* y respetando la estructura de directorios de la asignatura) deberá crear, en la herramienta *ModelSim*, un nuevo proyecto al que llamará *dff_ser*. Utilizando este proyecto, realice la descripción del circuito y guárdela en el fichero *dff_ser.vhd*. Como esqueleto puede usar el código de ejemplo en la siguiente página.

Observe como, en la lista de sensibilidad del proceso, sólo aparecen las señales *RST_an* y *CLK*. Ello es debido a que la salida de este circuito sólo puede cambiar de estado en coincidencia con un cambio en *RST_an* o en *CLK*, por ello estas son las únicas señales que pueden invocar la "ejecución" del proceso. Cualquier actividad en las entradas síncronas del circuito no produce un cambio inmediato en su salida (ese cambio sólo se producirá cuando llegue el flanco activo del reloj), por tanto esas señales síncronas no forman parte de la lista de sensibilidad del proceso.

A su vez, el proceso aparece modelado con una construcción *if-elsif*, para diferenciar el funcionamiento asíncrono debido a un nivel bajo en la señal de inicialización asíncrona *RST_an* (el *if*) del funcionamiento síncrono iniciado por un flanco de subida en *CLK* (el *elsif*). Observe como, tal y como está escrito el proceso, la señal *RST_an* tiene prioridad sobre el funcionamiento síncrono, tal y como se espera.

Su trabajo en esta parte se reduce, por tanto, a introducir el código necesario para modelar el funcionamiento del circuito: ante una activación de *RST_an* (código que debe aparecer entre el *if* y el *elsif*); y ante un flanco activo del reloj (código que debe aparecer entre el *elsif* y el *end if*).

```
library ieee;
  use ieee.std_logic_1164.all;

entity dff_ser is
  port(
    RST_an  : in      std_logic;
    CLK     : in      std_logic;
    SET_n   : in      std_logic;
    E       : in      std_logic;
    RST     : in      std_logic;
    D       : in      std_logic;
    Q       : buffer  std_logic
  );
end entity dff_ser;

architecture RTL of dff_ser is
begin
  process(RST_an, CLK)
  begin
    if RST_an = '0' then
      -- inserte aquí código para modelar el funcionamiento asíncrono

    elsif CLK'event and CLK = '1' then
      -- inserte aquí código para modelar el funcionamiento síncrono

    end if;
  end process;
end architecture RTL;
```

El funcionamiento asíncrono de este circuito es extremadamente simple: ante la activación de *RST_an* la salida debe pasar a valer 0.

El funcionamiento síncrono no es tan sencillo pero aun así es muy fácil de modelar en VHDL. Del análisis de la especificación del circuito se desprende que la entrada síncrona más prioritaria es *SET_n* (que es activa a nivel bajo). Una construcción tal como:

```
    if SET_n = '0' then
      -- inserte aquí código para modelar el funcionamiento al activarse SET_n

    elsif …
      -- inserte aquí código para modelar el funcionamiento al no activarse SET_n

    end if;
```

permitiría modelar el funcionamiento de esta señal. Dentro del *if* aparecería el código correspondiente al caso de que *SET_n* se active y en el *elsif* el código que modela el funcionamiento del circuito en caso contrario, es decir el código que modela el funcionamiento de *E*, *RST* y *D*.

La siguiente entrada síncrona, por prioridad, es *E*. Esta es una señal de *habilitación*, de manera que, dependiendo de su estado, el circuito realizará la función asociada a las entradas síncronas de menor prioridad (cuando *E* se active) o permanecerá inactivo (cuando *E* se desactive). Una manera sencilla de modelar este comportamiento puede ser:

```
    elsif E = '1' then
      -- inserte aquí código para modelar el funcionamiento cuando SET_n es inactivo
      -- y el flip-flop está habilitado por E

    end if;
```

donde el *elsif* es el mismo que el del ejemplo anterior.

Continuando con la misma estrategia el funcionamiento relativo a la señal *RST* puede modelarse mediante:

```
    if RST = '1' then
      -- inserte aquí código para modelar el funcionamiento cuando SET_n es
      -- inactivo, el flip-flop está habilitado por E y RST está activo

    else
      -- inserte aquí código para modelar el funcionamiento cuando SET_n es
      -- inactivo, el flip-flop está habilitado por E y RST está inactivo

    end if;
```

Escriba, por tanto, la descripción completa del *flip-flop*. Con esto termina la primera parte de esta actividad.

PARTE II. Simulación

Para simular el circuito deberá generar un *testbench* para el mismo. Este *testbench* contendrá dos procesos: uno para generar el reloj y un segundo para generar el resto de estímulos.

El proceso para la generación del reloj será:

```
CLOCK : process
begin
  CLK <= '1';
  wait for TCLK / 2;
  CLK <= '0';
  wait for TCLK / 2;
end process CLOCK;
```

donde `TCLK` es una *constante*. En el ejemplo anterior `TCLK` representa el valor 100 ns y se ha definido en la zona declaratoria de la arquitectura del *testbench* como:

```
constant TCLK : time := 100 ns;
```

En una actividad anterior se le han presentado ideas fundamentales a la hora de plantear el test de un sistema secuencial. Sin embargo, aprender el arte del diseño de *testbenches* es una tarea que requiere muchas horas de práctica y para la que existen una serie de criterios prácticos que permiten juzgar la bondad de un determinado *testbench*. Entre estos criterios se pueden citar:

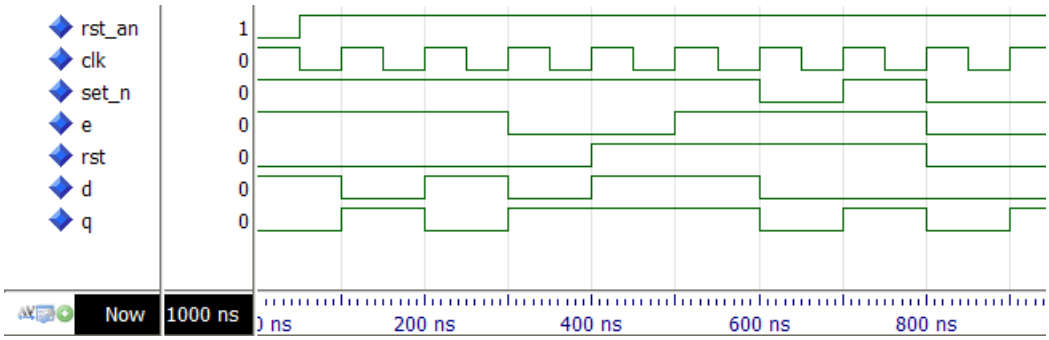
1. Todas las entradas y salidas individuales del circuito (y a veces también todas sus señales internas) deben cambiar de estado (a veces el criterio es más estricto y se exige que cambien al menos una vez de 0 a 1 y al menos otra vez de 1 a 0). En aquellos circuitos con señales de tres estados también debe verificarse que las señales en cuestión entran y salen del tercer estado (tanto desde como hacia un 1 como desde y hacia un 0). No puede asegurarse el correcto modelado de un circuito si alguna o algunas de sus entradas y salidas nunca cambian de estado durante la simulación. ¿Puede estarse seguro de que el modelado de un *flip-flop* es correcto si la secuencia de prueba que se aplica es tal que nunca pone la salida a 1?
2. El *testbench* debe ser capaz de excitar todas las líneas del código fuente VHDL que modela el funcionamiento del circuito. No puede asegurarse el correcto modelado si alguna línea del modelo no se ha excitado nunca y por tanto no se ha verificado el comportamiento que ella modela. ¿Puede estarse seguro de que el modelo del *flip-flop* anterior es correcto si la secuencia de test que se aplica nunca ejecuta la línea posterior a `if SET_n = '0' then`?
3. Cada decisión testeada en el modelo VHDL debe verificarse en todos los sentidos posibles. Es decir, no es suficiente con excitar una línea tal como:

   ```
   elsif E = '1' then
   ```

   sino que esa línea debe excitarse una vez siendo *E* igual a 1 y otra siendo *E* distinto de 1. ¿Puede estarse seguro de que el modelo del *flip-flop* anterior es correcto si la secuencia de test que se aplica nunca inhabilita al *flip-flop*?

Estos tres no son los únicos criterios que pueden seguirse a la hora de juzgar la adecuación del *testbench* de un circuito, pero si son más que suficientes para este curso. Debe entender que estos criterios no deben guiar su diseño del *testbench* (el diseño debe guiarse por consideraciones funcionales como las presentadas en una actividad anterior), sino que simplemente son una comprobación, *a posteriori*, de la bondad del *testbench*. El no cumplimiento de alguno de estos criterios suele ser indicativo de la no verificación de alguna funcionalidad del circuito.

En este caso se pide que genere un *testbench* para el circuito anterior que aplique la siguiente secuencia de test



Para realizar este *testbench* debe emplear construcciones del tipo *wait until* para ubicarse en aquellos instantes del tiempo en los que se producen los flancos de reloj, que son los instantes en los

que deben modificarse las entradas del circuito. Como esqueleto de partida para la realización de este *testbench* puede emplear el siguiente código:

```
library ieee;
  use ieee.std_logic_1164.all;

entity dff_ser_tb is
end entity dff_ser_tb;

architecture TB of dff_ser_tb is
  constant TCLK : time := 100 ns;
  signal RST_an : std_logic;
  signal CLK    : std_logic;
  signal SET_n  : std_logic;
  signal E      : std_logic;
  signal RST    : std_logic;
  signal D      : std_logic;
  signal Q      : std_logic;
begin
  DUV : entity work.dff_ser(RTL)
    port map(
      RST_an  => RST_an,
      CLK     => CLK,
      SET_n   => SET_n,
      E       => E,
      RST     => RST,
      D       => D,
      Q       => Q
    );

  CLOCK : process
  begin
    CLK <= '1';
    wait for TCLK / 2;
    CLK <= '0';
    wait for TCLK / 2;
  end process CLOCK;

  STM : process
  begin
    RST_an <= '0';
    SET_n <= '1';
    E <= '1';
    RST <= '0';
    D <= '1';
    wait until CLK'event and CLK = '1';
    RST_an <= '1';
    -- reset asíncrono terminado
    wait until CLK'event and CLK = '1';
    -- introduzca aquí el código restante

    wait;
  end process;
end architecture TB;
```

El fichero con el *testbench* debe llamarse *dff_ser_tb.vhd*. Guarde también el formato del visor de formas de onda en un fichero llamado *wave.do*.

Con esto termina la segunda parte de esta actividad.

PARTE III. Prueba en la placa MAX1000

Para probar el circuito en la placa MAX1000 deberá proceder a la compilación del modelo *dff_ser.vhd* mediante la herramienta *Quartus Prime*. La asignación de pines que debe usar es:

Circuito: *dff_ser*

Señal (en *dff_ser*)	Recurso (en placa)	*Pin* (en chip)
RST_an	*USER_BTN*	PIN_E6
CLK	*CLK_12M*	PIN_H6
SET_n	*SW4*	PIN_E3
E	*SW3*	PIN_D1
RST	*SW2*	PIN_C1
D	*SW1*	PIN_C2
Q	*LED1*	PIN_A8

Una vez volcado el diseño sobre la placa realice su verificación aplicando (mediante pulsadores y *switches*) una secuencia de test similar a la dada en el cronograma anterior.

Finaliza aquí la tercera y última parte de esta actividad.